

LECTURE 2 — OPEN-SOURCE ROBOTS

STANDARD MODELS, PLUGINS AND PACKAGES, OFF THE SHELF

Topics: The catalogue · A robot in the world · Sensors · Odometry · Motors · Control · Teleop · RViz · Lab 1

TODAY

- **The catalogue** — where robots come from, and what can be wrong with one
- **One robot, closely** — what it shows you, what it publishes, what it is made of
- **Plugins** — the only part of a world that runs code
- **Odometry** — where the robot thinks it is
- **Motors** — one plugin for a car, five for a drone
- **Control** — from the command line, then from the keyboard
- **Lab 1** — your first graded work

THE CATALOGUE

SOMEBODY ALREADY BUILT YOUR ROBOT



OFF THE SHELF

- **Fuel** — an online library of models and worlds, at `app.gazebo-sim.org/fuel`
- Each model has an **owner**, a **licence** and a **version number**
- The `<>` button on a model's page gives you the exact snippet to paste
- Real machines are there: Husky, Spot, TurtleBot, PX4 x500, X3 and X4 quadcopters
- **You will not build a robot today.** Making one from scratch is the next lecture

gazebo-sim.org/docs/harmonic/fuel_insert · app.gazebo-sim.org/fuel



TWO WAYS IN

Put the URL in your world

```
<include>
  <name>husky</name>
  <pose>0 0 0.2 0 0 0</pose>
  <uri>https://fuel.gazebo.org/1.0/
    OpenRobotics/models/
    COSTAR_HUSKY_SENSOR_CONFIG_1</uri>
</include>
```

Gazebo fetches it the first time you run, then keeps it. Your world file stays short.

- The URL is the one thing you need either way. The host is `fuel.gazebo.org` — `app.` is the web page
- **Download when you need to read the model.** The joint names, the wheel radius and the sensor names are all inside `model.sdf`, and there is nowhere else to get them

gazebo.org/docs/harmonic/fuel_insert · github.com/gazebo/gz-fuel-tools

Fetch it first, then read it

```
$ gz fuel download -u "https://fuel.\
gazebo.org/1.0/OpenRobotics/models/\
COSTAR_HUSKY_SENSOR_CONFIG_1"
$ ls ~/.gz/fuel/.../11/
model.config  model.sdf
meshes/      materials/
```

The same files, but now **on your disk, before you run.**

POTENTIAL ISSUES WITH A MODEL

The catalogue tells you the licence and the download count. It does not tell you which simulator the model was written for. What to look at before you commit to one:

#	CHECK	WHAT IT MEANS
1	<code>model.config</code> → <code><sdf version="..."></code>	1.4–1.6 is old enough to look closely
2	the model's last-modified date	untouched for years
3	<code><material><script></code> with no <code><uri></code> inside	materials written for the old simulator — this one takes the whole world down
4	<code><plugin filename="libgazebo_ros_..."></code>	a plugin for a simulator we do not run
5	meshes that are very large	a slow first load, and a slow every-frame after that
6	the console, on first load	<code>gz sim -s -r world.sdf -v 4</code>

sdformat.org/spec · gazebo-sim.org/docs/harmonic/comparison

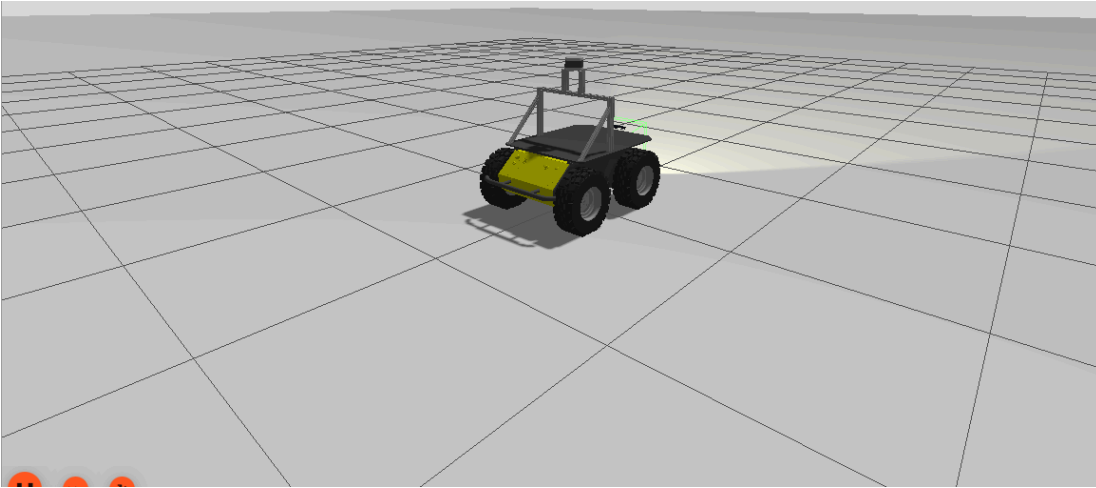


ONE ROBOT, CLOSELY

THE HUSKY, FROM THE OUTSIDE IN



THIS IS THE ROBOT



The Husky in our own world: a ground plane and one `<include>` .

- A Clearpath **Husky** — a four-wheeled outdoor robot, used for real
- The yellow body, four wheels, and a mast carrying the sensors
- It arrived from the catalogue as **one line** in our world file
- Right now it can do exactly nothing: it sits there and falls over if you push it
- Everything else in this lecture is about what you add to that

 app.gazebosim.org/OpenRobotics/fuel/models/COSTAR_HUSKY_SENSOR_CONFIG_1

WHAT IT CAN SENSE

SENSOR	TYPE	RATE	WHAT IT GIVES YOU
camera_front	rgb_camera	20 Hz	a colour picture, and a distance for every pixel
front_laser	gpu_lidar	10 Hz	distances all around, at many angles
imu_sensor	imu	50 Hz	how fast it is turning, and what it is being accelerated by

- **A lidar is a spinning laser.** It measures how far away things are, at each angle it points
- **An IMU** is the thing in your phone that knows which way up it is — rates of turn, and acceleration
- These came with the model. You did not choose them, and you cannot add to them without editing somebody else's file
- **⚠ Sensors only produce data if the world loads their plugin.** Leave it out and they are silent, with no complaint

WHAT IT PUBLISHES

```
$ gz topic -l
/world/husky_drive/model/husky/link/base_link/sensor/camera_front/image
/world/husky_drive/model/husky/link/base_link/sensor/camera_front/depth_image
/world/husky_drive/model/husky/link/base_link/sensor/camera_front/points
/world/husky_drive/model/husky/link/base_link/sensor/camera_front/camera_info
/world/husky_drive/model/husky/link/base_link/sensor/front_laser/scan
/world/husky_drive/model/husky/link/base_link/sensor/imu_sensor/imu
```

TOPIC ENDS IN	MESSAGE	WHAT IS IN IT
/image · /depth_image	Image	the colour picture · the same picture, distances instead of colours
/points	PointCloudPacked	those distances as points in space
/camera_info	CameraInfo	the lens numbers needed to turn pixels into metres
/scan	LaserScan	a list of distances, the starting angle, the step between them
/imu	IMU	turn rates and accelerations

- The name is built from **world / model / link / sensor**. Copy it from `gz topic -l`; never type it

WHAT IT IS MADE OF

ELEMENT	WHAT IT IS	WHY IT IS THERE
<code><model></code>	one named object — a robot, a table	the thing you include, move and address
<code><link></code>	one solid part — the body, one wheel	the thing physics moves
<code><joint></code>	a connection between two parts	a wheel and its axle · a door and its hinge · two parts held together so they never move apart
<code><inertial></code>	the part's mass, and how that mass is spread out	physics needs both to work out how it moves
<code><collision></code>	the shape that touches things	kept simple: a box or a cylinder
<code><visual></code>	the shape you see	the detailed mesh
<code><sensor></code>	a camera, a lidar, an IMU, on a part	makes data; moves with the part it sits on
<code><plugin></code>	code attached to something	the only thing here that <i>does</i> anything

 sdformat.org/spec?ver=1.11&elem=model



THE BODY

```
<link name="base_link">
  <inertial>
    <mass>46.064</mass>          <!-- kilograms -->
    <inertia>                    <!-- moment of INERTIA, in kg·m² -->
      <ixx>0.6022</ixx>          <!-- how hard it is to spin about X -->
      <iyy>1.7386</iyy>          <!-- ...about Y -->
      <izz>2.0296</izz>          <!-- ...about Z -->
    </inertia>
  </inertial>
  <collision name="c"><geometry><box><size>0.99 0.57 0.28</size></box></geometry></collision>
  <visual name="v"><geometry><mesh><uri>meshes/base_link.dae</uri></mesh></geometry></visual>
</link>
```

-  **Moment of inertia, not moment of force.** A moment of force is a torque, measured in N·m, and it is something you *apply*. This is a property of the object: kg·m², how much it resists being spun
- **Mass** decides how hard it is to *push*; **inertia**, how hard it is to *turn*. A long body resists turning more than a compact one of the same mass — hence three different numbers
- Collision and visual differ on purpose: contacts are worked out against the plain box; the mesh is only drawn

 sdformat.org/spec?ver=1.11&elem=link



LEAVE THE MASS OUT AND NOTHING COMPLAINS

```
# the same 10 N push, gravity switched off  
mass written as 10 kg    → moved 74 m  
no <inertial> at all     → moved 774 m
```

- The part does **not** disappear, and the simulator prints **no warning**
- SDF quietly fills in a **default of 1 kg**. Ten times too light here — and for this Husky's real body, **forty-six times** too light
- So the robot loads, drives, and is wrong: every push, every collision, every stop is far too effective

A WHEEL AND ITS AXLE

```
<joint name="front_left_wheel_joint" type="revolute">
  <parent>base_link</parent>          <!-- what it hangs from -->
  <child>front_left_wheel_link</child>  <!-- what turns -->
  <axis>
    <xyz>0 1 0</xyz>                  <!-- it turns about Y -->
    <limit><lower>-1e16</lower><upper>1e16</upper></limit>  <!-- no stop: it is a wheel -->
  </axis>
</joint>
```

- `revolute` turns, like a door on its hinge · `prismatic` slides, like a drawer · `fixed` holds two parts so they never move apart
- **There is no motor here.** A joint says a part *may* move. It does not make it move
- **Remember these four names** — `front_left`, `front_right`, `rear_left`, `rear_right`. The drive plugin will ask for them by name, and nothing else knows them

AND NOT ONE LINE OF CODE

```
$ grep -c "<plugin" model.sdf      # the Husky
0
```

- Body, wheels, axles, sensors — and **nothing that acts**
- No motor, no controller, no ROS. A model off the shelf is **shape and sensors**
- Making it move is a decision of **your world**, not of the model — which is why `<include>` is allowed to contain `<plugin>` children
- So the next question is: what is a plugin, and what can it do?

PLUGINS

THE ONLY PART OF A WORLD THAT RUNS CODE



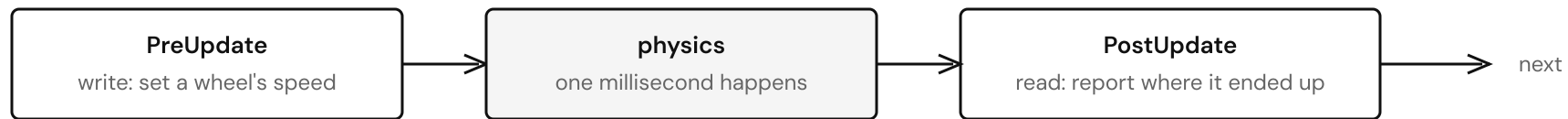
A PLUGIN IS A LIBRARY

```
$ gz plugin -i -p ../gz-sim-8/plugins/libgz-sim-diff-drive-system.so
* Found 1 plugin in library file:
  - gz::sim::v8::systems::DiffDrive
* Found 4 interfaces in library file:
  - gz::sim::v8::System
  - gz::sim::v8::ISystemConfigure      # called once, when the world loads
  - gz::sim::v8::ISystemPreUpdate      # called every step, BEFORE physics
  - gz::sim::v8::ISystemPostUpdate     # called every step, AFTER physics
```

- A plugin is a **.so file** on disk. `filename="gz-sim-diff-drive-system"` in your SDF names this file
- Around **140** of them ship with Gazebo — every plugin in this lecture is one of them
- The server loads it **into its own process** and calls it at the moments listed above

BEFORE PHYSICS, AFTER PHYSICS

The simulator repeats one small step, a thousand times per simulated second. A plugin can ask to be called at two moments of it:



- **PreUpdate** is the only chance to change anything. Physics has not run yet, so what you write here is what the world will obey
- **PostUpdate** is read-only. The step has already happened; you can look and report, not alter

THE PLUGIN HAS	SO IT IS ONE THAT	EXAMPLE
PreUpdate only	acts, and tells you nothing	a motor · a force
PreUpdate and PostUpdate	acts and reports	a drive that also says where it went
PostUpdate only	watches	a logger

PLUGIN OR NODE?

	GAZEBO PLUGIN	ROS 2 NODE
what it is	a library inside the simulator	its own program
when it runs	every physics step	on its own schedule, when messages arrive
what it can reach	the world itself	only messages
use it when	you must change physics : motors, forces, sensors	you write logic : perception, planning, control
examples	the drive, the motors, the odometry publisher	your own node · <code>teleop_twist_keyboard</code> · Nav2
in this course	you reuse them	you write them

- A node cannot set a wheel's speed inside the simulator. It can only send a message that a plugin acts on
- That message crossing the boundary is what the rest of this lecture is about

 github.com/gazebo/gz-sim/tree/gz-sim8/src/systems



MAKING IT MOVE

THE MODEL HAS NO MOTOR. YOU GIVE IT ONE



GIVING THE HUSKY A MOTOR

```
<include>
  <name>husky</name>
  <uri>https://fuel.gazebosim.org/1.0/OpenRobotics/models/COSTAR_HUSKY_SENSOR_CONFIG_1</uri>

  <plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
    <left_joint>front_left_wheel_joint</left_joint>      <!-- the names from model.sdf -->
    <left_joint>rear_left_wheel_joint</left_joint>        <!-- listed twice: four-wheel drive -->
    <right_joint>front_right_wheel_joint</right_joint>
    <right_joint>rear_right_wheel_joint</right_joint>
    <wheel_separation>0.5708</wheel_separation>          <!-- metres, left wheel to right wheel -->
    <wheel_radius>0.1651</wheel_radius>                  <!-- metres -->
  </plugin>
</include>
```

- The plugin goes **inside the** `<include>` — you are not editing the model, you are adding to your own world
- Both numbers are **read out of** `model1.sdf`. Leave them out and you get 1.0 m and 0.2 m instead: the robot still drives, and its odometry lies to you all semester

 gazebosim.org/docs/harmonic/moving_robot

WHAT YOU SEND IT: A `Twist`

```
$ ros2 interface show geometry_msgs/msg/Twist
Vector3  linear      # a straight-line speed, in metres per second
    float64 x        #   forward, positive ahead
    float64 y        #   sideways, positive to the left
    float64 z        #   up, positive upward
Vector3  angular     # a turning speed, in radians per second
    float64 x        #   roll – tipping left and right
    float64 y        #   pitch – nose up and down
    float64 z        #   yaw – turning left, positive anticlockwise
```

- **Six numbers, and nothing else.** No timestamp, no sender, no frame — `TwistStamped` is the version that adds those
- It is a **speed, not a position and not a distance**: "go at half a metre per second", never "go half a metre"
- A Husky can only use **two of the six** — `linear.x` and `angular.z`. It cannot fly, and it cannot slide sideways

 github.com/ros2/common_interfaces/blob/jazzy/geometry_msgs/msg/Twist.msg



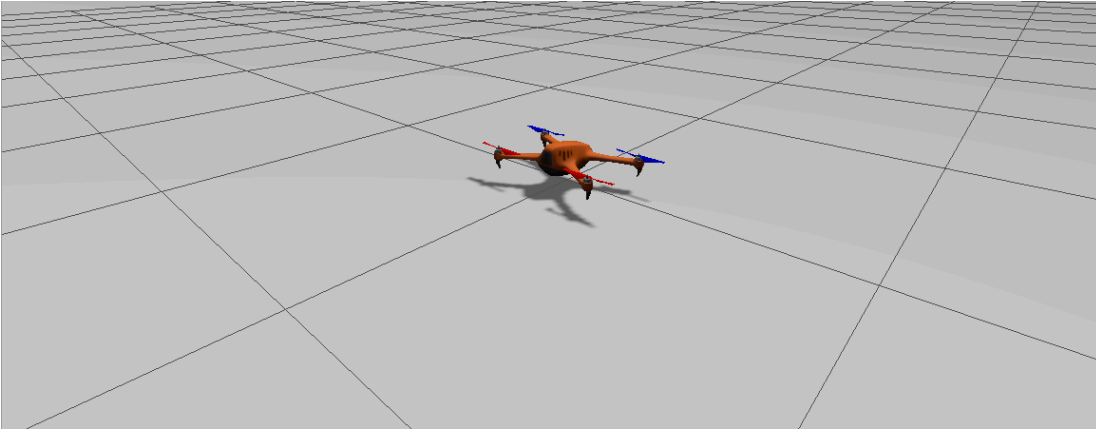
WHAT THE DRIVE DOES WITH IT

- It is **the controller and the motor in one**
- It takes the forward speed and the turn rate out of the `Twist`, and works out **how fast each side of the robot must spin**
- That answer depends on three things: the **speed asked for**, the **turn rate asked for**, the **wheel separation** and the **wheel radius** — the two numbers you put in the world. Turn faster, or set the wheels further apart, and the two sides differ more
- **Where does that arithmetic happen?** Inside the plugin, inside the simulator, on every single step. Not in your node, and not in the message
- Then it **sets each wheel joint's speed directly** — not a force, a speed
- So it is a **perfect motor**: unlimited torque, no delay, no slipping clutch, no battery going flat. The robot you drive today is easier than any real one

 github.com/gazebosim/gz-sim/blob/gz-sim8/src/systems/diff_drive/DiffDrive.cc



THE SAME QUESTION, FOR A DRONE



X3 in our own world. Like the Husky: no plugins in the model.

```
$ grep -c "<plugin" model.sdf    # the X3
0
```

- Same story — the model is shape and nothing more
- But where the Husky needed **one plugin for the whole robot**, here a motor has to be added **to each propeller separately**
- Four blocks, identical except for the propeller and which way it spins
- And a motor is only a motor: it does not take "go forward"

WHAT YOU SEND A ROTOR

```
$ ros2 interface show actuator_msgs/msg/Actuators
std_msgs/Header header      # when this command was issued
float64[] position          # where each actuator should sit – rad for something that turns,
                             #   m for something that slides. Empty here: propellers do not hold a position
float64[] velocity          # how fast each actuator runs – rad/s for a propeller.
                             #   THIS is the one we use: one number per rotor, in the order declared
float64[] normalized        # anything that is not a position or a speed, scaled to -1 ... 1
                             #   (a servo at half travel, a throttle at 70%). Empty here
```

```
$ ros2 topic pub /x3/command/motor_speed actuator_msgs/msg/Actuators \
  '{velocity: [600, 600, 600, 600]}'
```

- **Three lists, and you fill in the one that fits your hardware.** A propeller has a speed, so `velocity`
- **Nothing in that message says "up".** You are spinning propellers and letting physics decide

TWO MOTORS, TWO LANGUAGES

	HUSKY'S DRIVE	THE DRONE'S MOTORS
message it accepts	<code>geometry_msgs/msg/Twist</code>	<code>actuator_msgs/msg/Actuators</code>
what the numbers mean	how the robot should move	how fast each propeller spins
units	m/s and rad/s	rad/s, one per rotor
how many numbers	6, of which it uses 2	4 — one per rotor
who does the arithmetic	the plugin , for you	you do
how many plugins	one, for the whole robot	one per rotor
can you steer with it	yes	no — only up

- This is the whole difference, and it is not really about wheels against propellers
- One plugin was written to accept an **intention**. The other accepts a **command to a piece of hardware**
- Spin the four evenly and it climbs. Spin them unevenly and it tips — and nothing puts it back

**THE HUSKY NEEDS ONE PLUGIN. THE DRONE NEEDS FIVE.
WHY?**



BECAUSE FOUR OF THEM ARE MUSCLES

PLUGIN	HOW MANY	MESSAGE IT GETS	WHAT IT DOES
motor model	one per rotor — four	<code>Actuators</code> — one spin speed	turns that speed into a push upward and a twist about the shaft. Knows nothing about the aircraft
velocity control	one	<code>Twist</code> — an intention	reads how the aircraft is tilted, works out what all four rotors must do about it, and writes those four speeds. Every step

- The fifth plugin is the **brain**, and the Husky's drive had one built in all along
- For wheels the arithmetic is exact and finished, and the ground holds the robot level for free
- A quadrotor has **four numbers to control six ways of moving**, and it is **unstable**: tilt it a little and it tips further by itself. Nothing holds it level, so something must watch and correct, continuously
- With the brain, the drone accepts the same `Twist` as the Husky. Without it, you are spinning propellers by hand

 gazebo.org/api/sim/8/classgz_1_1sim_1_1systems_1_1MulticopterVelocityControl.html

DRIVING IT FROM ROS 2

ACROSS THE BOUNDARY



THE BRIDGE, WRITTEN DOWN

```
# config/husky.yaml – our own file
- ros_topic_name: "/cmd_vel"           # the short name we want in ROS 2
  gz_topic_name: "/model/husky/cmd_vel" # the long name the simulator uses
  ros_type_name: "geometry_msgs/msg/Twist"
  gz_type_name: "gz.msgs.Twist"
  direction: ROS_TO_GZ                 # commands go IN

- ros_topic_name: "/odom"
  gz_topic_name: "/model/husky/odometry"
  ros_type_name: "nav_msgs/msg/Odometry"
  gz_type_name: "gz.msgs.Odometry"
  direction: GZ_TO_ROS                 # measurements come OUT
```

- One entry per topic, and **only the topics you actually use** — the bridge copies every message it is told about
- The two names are independent, so **the bridge renames**. That is how an awkward simulator topic becomes a plain

/cmd_vel

 github.com/gazebo-sim/ros_gz/blob/jazzy/ros_gz_bridge/README.md

EVERY TOPIC THE LAB NEEDS

ROBOT	ROS 2 TOPIC	TYPE	SIMULATOR TOPIC	DIRECTION
Husky	/cmd_vel	geometry_msgs/msg/Twist	/model/husky/cmd_vel	in
Husky	/odom	nav_msgs/msg/Odometry	/model/husky/odometry	out
drone	/cmd_vel	geometry_msgs/msg/Twist	/X3/gazebo/command/twist	in
drone	/odom	nav_msgs/msg/Odometry	/model/x3/odometry	out

- **On the ROS 2 side both robots use the same two names.** That is the bridge doing its job, and it is what lets one keyboard program drive either of them
- On the simulator side the drone's two names differ in more than wording: commands go to /X3/..., measurements come from /model/x3/...

DRIVING IT

```
$ ros2 run ros_gz_bridge parameter_bridge --ros-args -p config_file:=config/husky.yaml
[INFO] Creating ROS->GZ Bridge: [/cmd_vel -> /model/husky/cmd_vel]
[INFO] Creating GZ->ROS Bridge: [/model/husky/odometry -> /odom]

$ ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist '{linear: {x: 0.5}}'
$ ros2 topic echo /odom --field pose.pose.position
```

- Half a metre per second for six seconds should leave you about three metres further on. **Check that it does** — this is the first moment the whole chain is under test
- `-r 10` repeats ten times a second. **Without it you send one message — and one is enough:** the drive keeps the last thing it was told, indefinitely
- So **stopping is something you have to do**, not something that happens by itself:

```
$ ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist '{}' # all zeros
```

ODOMETRY

WHERE THE ROBOT THINKS IT IS



WHAT ODOMETRY IS

Odometry is working out where you are by adding up how you have moved.

- The wheels turned this much → the robot went this far → it must be *there*
- It needs **nothing outside the robot**: no satellites, no map, no beacons. Free, and smooth
- And it is **wrong, more so every minute**. A wheel slips, a radius is off by a millimetre — each small error is added to the total and **never taken back**
- Drive in a circle and come back: odometry will not say you are where you started

A word on names. This is **wheel odometry** — counted from the wheels. **Visual odometry** answers the same question from a camera, and **lidar odometry** from a lidar. Same question, different sensor, quite different maths. We use the wheels today.

nav_msgs/msg/Odometry, FIELD BY FIELD

FIELD	TYPE	WHAT IT IS
header.stamp	time	when this was true. In simulation this is simulation time , not the wall clock
header.frame_id	text	what the position is measured against — the place the robot started from
child_frame_id	text	which part is moving. The speed below is measured in <i>this</i> part's own axes
pose.pose.position	x, y, z metres	where it is
pose.pose.orientation	quaternion	which way it faces — four numbers rather than three angles, because three angles jam at the poles
pose.covariance	36 numbers	how sure it is about the position, as a 6×6 table
twist.twist.linear	x, y, z m/s	how fast it is moving right now , forward / sideways / up
twist.twist.angular	x, y, z rad/s	how fast it is turning right now
twist.covariance	36 numbers	how sure it is about the speed

- **Two frames on purpose:** the position is measured against the world, the speed against the robot itself

 github.com/ros2/common_interfaces/blob/jazzy/nav_msgs/msg/Odometry.msg

THE TWO COVARIANCE TABLES

```
$ ros2 topic echo /odom --field pose.covariance --once  
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. ... ]
```

- **Covariance is the sender saying how sure it is.** Six quantities — x, y, z and the three rotations — so a 6×6 table: the diagonal is "how uncertain is each one", the rest is "do their errors move together"
- Here it is **all zeros, because nobody worked one out.** The simulator's own message has no such field at all; the ROS message insists on having one, so the bridge fills in zeros
- **⚠ Zero does not mean perfect.** It means *no information*. Something that genuinely estimates its own uncertainty fills these in — that is a later lecture
- **The plain version:** `geometry_msgs/msg/Pose` is position and orientation, and nothing else. Use it when all you need to say is where something is

TWO WAYS TO GET IT

	THE DRIVE PLUGIN'S OWN	A SEPARATE ODOMETRY PLUGIN
where it comes from	the drive already knows what it told the wheels, so it adds that up	it watches the model move and reports what it sees
what it costs you	nothing — it is already there	one more plugin in the world
flat or 3-D	flat: <code>z</code> is always 0	you choose 2 or 3
when you need it	any wheeled robot with a drive	a drone — and anything whose motor reports nothing

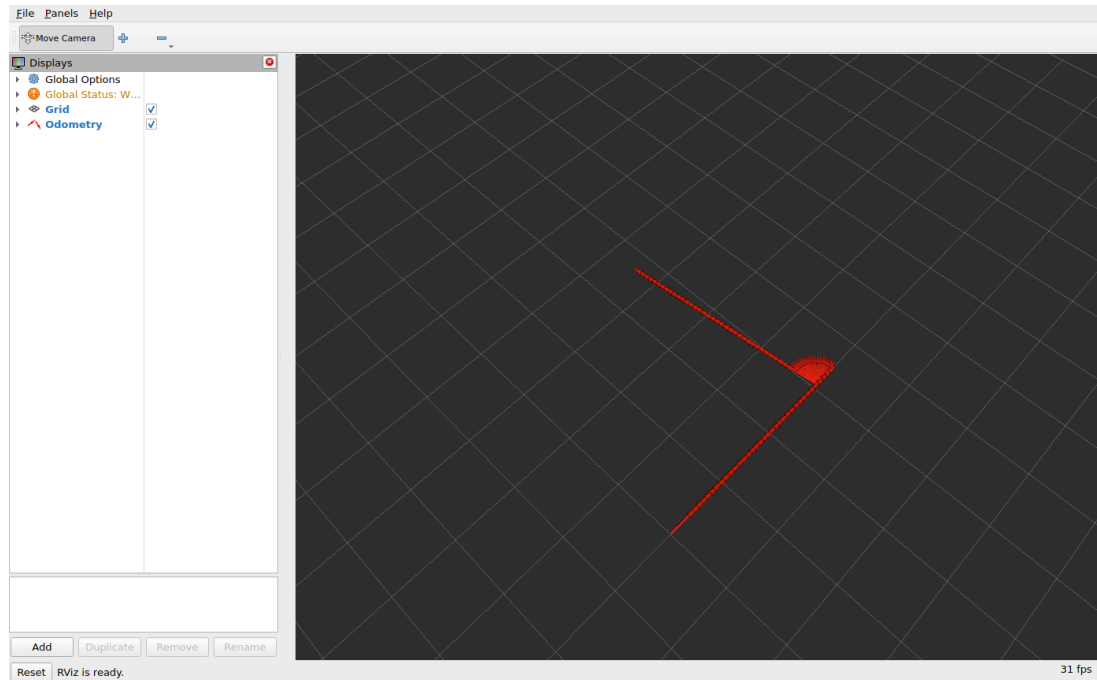
- **Why two?** Because not every robot has a drive plugin. A bare motor has no `PostUpdate`, so it cannot report anything, and something else has to
- That is why a wheeled robot hands you odometry for nothing, and a drone does not

RVIZ

LOOKING AT WHAT THE ROBOT SAYS



WHAT RVIZ IS

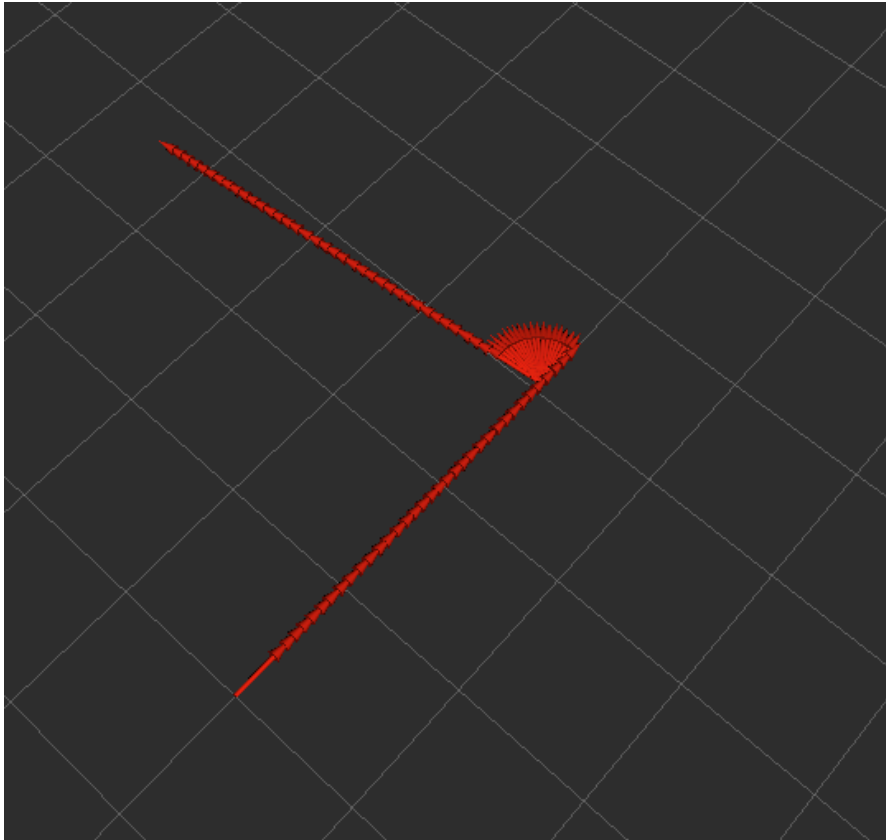


RViz: what to draw on the left, the drawing on the right.

- It draws **data that is on ROS topics**. It is not a simulator, and it is not the Gazebo window
- You add one **display** for each topic you want to see
- Setting it up is three clicks: say what the drawing is measured against, add the display, done
- ⚠ It shows only what is **published**. A topic nobody bridged does not appear, and RViz will not tell you why

docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html · docs.clearpathrobotics.com/docs/ros/tutorials/rviz

THE PATH, DRAWN



One run: straight, one 90° turn, straight.

- The **Odometry** display puts one arrow at every position the robot reported, and keeps the last few hundred
- The trail *is* the path it drove
- The dense fan is the turn: the robot spun on the spot, so the positions piled up
- Nothing here is a map, and nothing was measured from outside. This is the robot's own account of itself — **with all the drift that implies**

WHAT ELSE YOU CAN PUT IN IT

DISPLAY	TOPIC TYPE	YOU WILL USE IT FOR
Odometry · Path	Odometry · Path	where the robot has been — arrows, or a clean line
LaserScan	LaserScan	the lidar, as a ring of points around the robot
PointCloud2	PointCloud2	anything in 3-D: the depth camera, a 3-D lidar
Image · Camera	Image	the camera picture, in a panel of its own
Map	OccupancyGrid	the map, once something is building one
Marker · MarkerArray	Marker	anything you want — arrows, lines, text your own node draws
RobotModel · TF	—	the robot's own body and its parts. Needs more than we have today

-  There is **no display for an IMU**. People look for one every year — read it with `ros2 topic echo`, or plot it
- **Markers are the escape hatch:** when nothing fits, your node publishes shapes and RViz draws them

 docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html



teleop_twist_keyboard

TWO HUNDRED LINES THAT DRIVE ANYTHING



HOW IT TAKES A KEY

```
def getKey(settings):  
    tty.setraw(sys.stdin.fileno())      # raw mode: give me keys at once, don't wait for Enter  
    key = sys.stdin.read(1)             # waits here until exactly one character arrives  
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)  # put the terminal back  
    return key
```

- Normally a terminal hands your program a **whole line**, once you press Enter. Raw mode switches that off
- The settings are saved at the start and put back after **every single key** — otherwise a crash would leave your shell unusable
- Two things follow: the terminal must be **the window you are typing into**, and this program **cannot be started from a launch file**, because a launch file gives it no terminal to read from

 github.com/ros2/teleop_twist_keyboard/blob/jazzy/teleop_twist_keyboard.py



WHAT THE KEYS MEAN

```
moveBindings = {                                # key : (x, y, z, th)
    'i': ( 1,  0,  0,  0),                       # forward
    'j': ( 0,  0,  0,  1),                       # turn left
    'l': ( 0,  0,  0, -1),                       # turn right
    ',': (-1,  0,  0,  0),                       # back
    'J': ( 0,  1,  0,  0),                       # SHIFT: slide left without turning
    't': ( 0,  0,  1,  0),                       # UP
    'b': ( 0,  0, -1,  0),                       # DOWN
}
speedBindings = {'q': (1.1, 1.1), 'z': (.9, .9)}  # everything 10% faster / slower
```

- A plain lookup table. Lower case turns, **Shift** slides sideways, **t** and **b** go up and down
- **Any key that is not in the table sets all four to zero** — that is the stop button, and it is every other key on the keyboard
- Those **t** and **b** lines are about to matter

 github.com/ros2/teleop_twist_keyboard/blob/jazzy/teleop_twist_keyboard.py

HOW IT WRITES

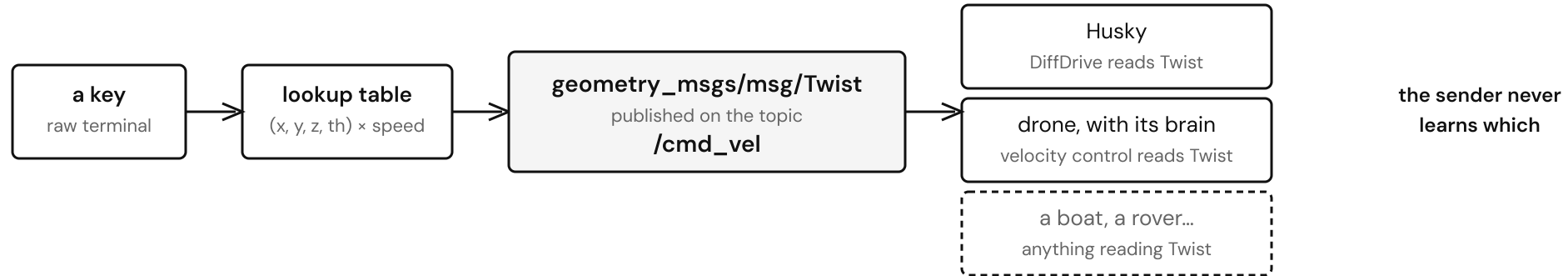
```
pub = node.create_publisher(TwistMsg, 'cmd_vel', 10)      # RELATIVE name – no leading slash
...
twist.linear.x  = x * speed                               # the table entry, scaled
twist.angular.z = th * turn
pub.publish(twist)
...
finally:
    pub.publish(TwistMsg())                               # all zeros, on the way out
```

- `speed` starts at **0.5 m/s** and `turn` at **1.0 rad/s**; `q` and `z` scale them as you go
- ⚠ Those are **read-only parameters**: `ros2 param set` will refuse them. Set them when you start the program
- That `finally` is the difference between letting go of the keyboard and watching your robot leave

 github.com/ros2/teleop_twist_keyboard/blob/jazzy/teleop_twist_keyboard.py



WHY THIS DRIVES ANYTHING



- **A standard message** — `Twist` means the same thing to every mobile robot in ROS 2
- **A name you can change** — `cmd_vel` is written with no leading slash, so it can be pointed anywhere without touching the file
- **⚠ The limit:** it will not fly a drone that has only motors. Those read `Actuators`. Matching *types* is exactly how far this reaches

USING IT

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args --remap cmd_vel:=/my/topic
```

- The same program, unchanged, for the Husky and for a drone that has its brain: after the bridge has renamed things, **both read /cmd_vel**
- **i** drives the Husky forward · **t** takes the drone up. Same program, same message, different robot
- If your robot reads a different topic, **point the program at it** rather than editing it — that is what the remap is for
- The simulator and the bridge can be started however you like — a script, a launch file. **This one cannot:** it needs a terminal of its own

LAB 1

YOUR FIRST GRADED WORK



LAB 1 – WHAT YOU DO

Take a standard robot off the shelf, and drive it from ROS 2. Any model in the catalogue; wheels or rotors, your choice.

1. **Choose and check** — run the checks before you commit to a model
2. **Your own world** — a ground plane and an `<include>`, plus whatever plugin that model needs to move
3. **Bridge it, both ways** — commands in, odometry out. Only the entries you use
4. **Read the telemetry** — a small node that subscribes to `/odom` and reports where the robot is
5. **Drive it from the command line** — and check the distance against what you asked for
6. **Then from the keyboard** — `teleop_twist_keyboard`, with no changes to it
7. **Show the path** — in RViz, **and** as a plot drawn from `/odom`

You are given a repository with the skeleton in it. Work in that.

LAB 1 – WHAT IT SHIPS

IN THE README	WHAT GOES IN IT
Run	the exact commands, from zero, on a machine that is not yours
See	the path: a screenshot from RViz and a plot drawn from <code>/odom</code>
Numbers	the distance you asked for, the distance you got, and the difference
Compare	driving from the command line against driving from the keyboard — a paragraph, not an essay
<code>notes.txt</code>	what broke, what you tried, what is still unclear. It cannot lower your mark

- The README **is** the deliverable. In a year it is read by an employer, not by me
- Two things that will cost you time, so hear them now: the **first run needs the network**, and a model with problems will often not say so

LAB 1 – SEVEN FILES

```
my_fuel_lab/  
├─ package.xml  setup.py  setup.cfg  resource/...  # the usual Python-package wrapping  
├─ worlds/<name>.sdf      # ground plane, light, <include>, the movement plugin  
├─ config/bridge.yaml    # both directions, only the entries you use  
├─ <pkg>/odom_listener.py # ~30 lines: subscribe to /odom, print where you are  
├─ scripts/run.sh        # ONE script: simulator + bridge + listener  
├─ README.md             # Run · See · Numbers · Compare  
└─ notes.txt
```

- **No launch file.** A three-line `run.sh` does the same and needs nothing explained first
- **No saved RViz layout.** Three clicks set it up, and doing them by hand teaches more than a file nobody opens
- Both arrive later, the moment there are more processes than three

LAB 1 – IF YOU WANT MORE

Deliberately **not** in the hand-in — Lab 1 must fit in an evening. Pick one for the top mark:

STEP UP TO	WHAT IT GETS YOU
a mission node	~20 lines that drive a fixed sequence, instead of you typing commands. Your first replacement of a stock tool — and it makes two students' runs comparable
<code>ros2 bag</code>	record the drive; replay it into RViz with the simulator switched off
a launch file	start the simulator, the bridge and your node with one command
a saved RViz layout	for when three clicks stop being enough
the drone, steered	add the brain plugin and fly it with the same keyboard program

Nothing here is wasted: every row is a later lab, arriving early

WHAT'S NEXT

Lecture 3 — Building the Robot

Describing a robot yourself · parts, joints and frames · the robot that finally appears in RViz

 gazebo.org/docs/harmonic

